Nearest Neigh Classifiers:

⇒ instance based ML alg.
⇒ used for classification & regression tasks.
⇒ works by finding 'k' closest data points to a given i/p.

Steps:

1. choose a val for 'k' (no. of neighbours to consider)
2. calc the dist b/w new data point & all points in the dataset.
3. Sort & select the K closest data points (neighbours).
4. for classification: assign most common class among k neighb. (majority class val)
   for regression: take avg val of k-neighbs (avg class val).

dist metric:

1. euclidian dist (most common): Best for continuous numerical data

$$d(A,B) = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

2. Manhattan dist: best when features have grid-like properties (ex: chessboard)

3. Hamming dist: used for text classification or DNA sequence matching.

Advantages:

* Simple & easy to implement: does not req complex mathematical model.
* no training phase (lazy learning): KNN does not build a model, it just stores data.
* works well with small datasets: does not req lots of training data to work well.

disad

* requires proper choice of 'k': (affects accuracy): or it will lead to underfitting (small 'k') / overfitting (large 'k')
* struggles with imbalanced datasets: KNN may be biased to majority class (one class is larger than others)
* struggles — high-dimen data: (< accuracy).
* computationally expensive for large datasets:
  (since it calcs dist for all data, it might get slower for large datasets)

Methods to improve knn:

1. reduce dataset size:
   - ↳ reducing dataset size can speed up predictions.
   - ↳ computing dists becomes easier.
   - ↳ By using sampling of data method (dataset ⇒ subsets ⇒ computing).

2. Use efficient data strucs:
   - ↳ use efficient search algs (instead of brute force)
   - ↳ KDTree: K-dimen tree.
     Organizes data in hierarchial tree to reduce dist computation
   - ↳ Ball Tree: nested data partitioning
   - ↳ Hashing: used for high dimen data.
     uses hashing to quickly approximate NNs.

3. Feature selection:
   - ↳ irrelevant features increases computation time & decreases accuracy.
   - ↳ due to this KNN struggles with high-dimen spaces
   - ↳ filter methods: use techniques to remove unnecessary features.
   - ↳ embedded methods: identification of imp features.
   - ↳ This reduces mem usage & improves speed & accuracy.

4. Feature scaling:
   - ↳ KNN relies on dist calcs, features with large numeric ranges dominate smaller ones.
   - ↳ Min-Max Scaling: scales vals b/w 0 & 1.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

   - ↳ Z-score normalization: standardization.
     makes data centered at 0 with unit variance

$$x' = \frac{x - \mu}{\sigma}$$

5. Optimize 'K':
   - ↳ optimize the 'K' value
   - ↳ as choosing too small 'k' ⇒ overfitting (high variance)
     & ~ ~ large 'K' ⇒ underfitting (low variance)
   - ↳ use cross-validation to find best 'k' value.
   - ↳ use elbow method (plot accuracy v/s 'K' & choose best balance)
   - ↳ this prevents over/under fitting, leading to better generalization.

6. Use weighted KNN:

↳ all neighbors have equal importance, which is not always optimal
↳ assign higher weights to closer neighbors using inverse dist formula:

$$w = \frac{1}{d^2}$$

↳ as closer points contribute more to classification.
↳ improves accuracy, especially for imbalanced datasets.

7. Handle imbalanced datasets:

↳ if one class is much larger than others, KNN bias toward majority class.
↳ use 1. stratified sampling ⇒ ensure each class is represented fairly
2. Synthetic data generation ⇒ apply SMOTE to balance classes.
3. cost-sensitive learning ⇒ assign higher weigh to minority class neighbors.
↳ improves classification performance on minority classes.

8. Parallize Computation:

↳ Computing dists for all points is slow.
↳ Use 1. multithreading: to compute dists in 11th. eg: Numpy
2. GPU acceleration: for large-scale KNN computations. eg: TensorFlow
↳ significantly reduces computation time.

KNN Classifier Solved Example - 1

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

Sepal Length	Sepal Width	Species
5.2	3.1	?

Step 1: Find Distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

Distance (Sepal Length, Sepal Width) = 0.608

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608

KNN Classifier Solved Example - 1

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 2: Find Rank

Subscribe to Mahesh Huddar www.vtupulse.com

KNN Classifier Solved Example - 1

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 3: Find the Nearest Neighbor

If k = 1 – Setosa

If k = 2 – Setosa

If k = 5 – Setosa